

فصل ۷

مثال‌های همگامی

در فصل ۶، مسئله‌ی بخش بحرانی را مطرح کرده و بر چگونگی وقوع شرایط مسابقه موقعی که پردازه‌های همروند متعدد داده‌ها را به‌اشتراک می‌گرفتند تمرکز کردیم. چندین ابزار را بررسی کردیم که با پیشگیری از وقوع شرایط مسابقه مسئله‌ی بخش بحرانی را حل می‌کردند. این ابزار از راه‌حل‌های سخت‌افزاری سطح پایین گرفته (همچون سدهای حافظه و عملیات compare_and_swap()) تا ابزار سطح بالاتر روبه‌رشد (از قفل‌های انحصار متقابل تا سمافورها و مانیتورها) را شامل می‌شدند. همین‌طور، چالش‌های گوناگون طراحی برنامه‌هایی را که عاری از شرایط مسابقه هستند، اعم از خطرات عاقبت به‌خیری، همچون بن‌بست‌ها را بحث کردیم. در این فصل، ابزار مطرح شده در فصل ۶ را برای چندین مسئله‌ی کلاسیک همگامی به‌کار می‌بریم. همین‌طور، راهکارهای همگامی مورد استفاده در سیستم‌عامل‌های لینوکس، یونیکس، و ویندوز را بررسی کرده و جزئیات API هر دوی سیستم‌های جاوا و پوزیکس را بیان می‌کنیم.

اهداف فصل

- تشریح مسئله‌های همگامی بافر محدود، خوانندگان–نویسندگان، و فیلسوفان خوش‌خوراک.
- تبیین ابزار خاص استفاده شده توسط لینوکس و ویندوز به‌منظور حل مسئله‌های همگامی پردازه.
- نمایش چگونگی استفاده از پوزیکس و جاوا برای حل مسئله‌های همگامی پردازه.
- طراحی و توسعه‌ی راه‌حل‌هایی برای مسئله‌های همگامی پردازه با استفاده از APIهای پوزیکس و جاوا.

۷–۱ مسائل کلاسیک همگامی

در این بخش، تعدادی از مسائل همگامی را به عنوان نمونه‌هایی از رده‌ی بزرگ مسائل کنترل همروندی مطرح می‌کنیم. این مسائل تقریباً برای آزمون هر طرح همگامی پیشنهادی جدید استفاده می‌شوند. در راه‌حل‌های خود برای مسائل از سمافورها جهت همگامی استفاده می‌کنیم، چرا که به عنوان یک شیوه‌ی سنتی برای نمایش این قبیل راه‌حل‌هاست. با این وجود، پیاده‌سازی‌های واقعی این راه‌حل‌ها می‌توانند به جای سمافورهای دودویی از قفل‌های انحصار متقابل استفاده کنند.

۷-۱-۱ مسأله‌ی بافر محدود

مسأله‌ی بافر محدود در بخش ۶-۱ معرفی شد؛ این مسأله بیشتر برای نمایش قدرت بدویات همگامی استفاده می‌شود. در اینجا، ساختار کلی این طرح را بدون توجه به جزئیات پیاده‌سازی خاصی ارائه می‌دهیم. در همین راستا، در تمرین‌های انتهای فصل نیز یک پروژه‌ی برنامه‌نویسی مرتبط تدارک دیده‌ایم.

در مسأله‌ی ما، پردازه‌های تولیدکننده و مصرف‌کننده ساختمان‌داده‌های زیر را به اشتراک می‌گیرند:

```
int n;
semaphore mutex = 1;
semaphore empty = n;
semaphore full = 0;
```

فرض بر این است که انبار شامل n بافر است که هر یک قابلیت نگه‌داری یک عنصر را دارند. سمافور mutex، دسترسی انحصاری انبار بافر را فراهم می‌کند که در ابتدا به ۱ مقداردهی می‌شود. سمافورهای full و empty، به ترتیب، تعداد بافرهای پر و خالی را می‌شمارند. سمافور empty در ابتدا به n و سمافور full به ۰ مقداردهی می‌شوند.

کد پردازه‌ی تولیدکننده در شکل ۷-۱ و کد پردازه‌ی مصرف‌کننده در شکل ۷-۲ نشان داده می‌شوند. به تقارن میان تولیدکننده و مصرف‌کننده توجه کنید. می‌توانیم کد را این گونه تفسیر کنیم که تولیدکننده برای مصرف‌کننده، بافرهای پر تولید می‌کند یا این که، مصرف‌کننده برای تولیدکننده بافرهای خالی تولید می‌کند.

```
while (true) {
                . . .
        /* produce an item in next_produced */
                . . .
        wait(empty);
        wait(mutex);
                . . .
        /* add next_produced to the buffer */
                . . .
        signal(mutex);
        signal(full);
}
```

شکل ۷-۱ ساختار پردازه‌ی تولیدکننده.

۷-۱-۲ مسأله‌ی خوانندگان-نویسندگان

فرض کنید که یک پایگاه‌داده مابین چندین پردازه‌ی همروند به اشتراک گذاشته می‌شود. برخی از این پردازه‌ها ممکن است بخواهند فقط پایگاه‌داده را بخوانند، در حالی که بقیه ممکن است

بخواهند پایگاه‌داده را به روز کنند (یعنی، بخوانند و بنویسند). با خطاب پردازه‌های نوع اول، به نام ***خوانندگان*** و پردازه‌های نوع دوم، تحت عنوان ***نویسندگان***، این پردازه‌ها را از هم تمییز می‌دهیم. واضح است که اگر دو خواننده همزمان به داده‌های مشترک دسترسی داشته باشند، هیچ اتفاق ناگواری نخواهد افتاد. با این وجود، اگر یک نویسنده و پردازه‌ی دیگری (خواننده و یا نویسنده) همزمان پایگاه‌داده را دستیابی کنند، بروز مشکل حتمی است.

```
while (true) {
        wait(full);
        wait(mutex);
                . . .
        /* remove an item from buffer to next_consumed */
                . . .
        signal(mutex);
        signal(empty);
                . . .
        /* consume the item in next_consumed */
                . . .
}
```

شکل ۷-۲ ساختار پردازه‌ی مصرف‌کننده.

برای تضمین این که چنین مشکلاتی پیش نمی‌آید، لازم داریم که دسترسی پایگاه‌داده‌ی مشترک را توسط نویسندگان در زمان نوشتن انحصاری کنیم. این مسأله‌ی همگامی به **مسأله‌ی خوانندگان-نویسندگان** معروف است. به خاطر قدمتی که این مسأله دارد، تقریباً برای آزمون هرگونه بدویات همگامی جدید استفاده می‌شود.

مسأله‌ی خوانندگان-نویسندگان گونه‌های متعددی دارد که تمامی آنها مشمول الویت هستند. ساده‌ترین آن که به مسأله‌ی *اول* خوانندگان-نویسندگان معروف است، لازم دارد که هیچ خواننده‌ای منتظر نماند، مگر این که نویسنده‌ای پیش از آن، مجوز استفاده از شئ مشترک را کسب کرده باشد. به عبارت دیگر، هیچ خواننده‌ای نباید به این خاطر که نویسنده‌ای منتظر است، به انتظار اتمام کار خوانندگان دیگر بماند. مسأله‌ی *دوم* خوانندگان-نویسندگان لازم دارد که، یک نویسنده به محض آمادگی، عملیات نوشتن خود را در اسرع وقت انجام دهد. به عبارت دیگر، اگر نویسنده‌ای به انتظار دسترسی شیئی باشد، هیچ خواننده‌ی جدیدی نمی‌تواند خواندن خود را شروع کند.

راه‌حل این دو مسأله ممکن است منجر به قحطی‌زدگی شود. در مورد اول، امکان دارد که نویسندگان قحطی‌زده شوند؛ در مورد دوم، ممکن است خوانندگان با قحطی مواجه شوند. به همین دلیل، گونه‌های دیگری از مسأله مطرح شده‌اند. در ادامه، راه‌حلی برای اولین مسأله‌ی خوانندگان-نویسندگان ارائه می‌دهیم.

در راه‌حل مسأله‌ی اول خوانندگان–نویسندگان، پردازه‌های خواننده ساختمان‌داده‌های زیر را به اشتراک می‌گذارند:

```
semaphore rw_mutex = 1;
semaphore mutex = 1;
int read_count = 0;
```

سمافورهای mutex و rw_mutex در ابتدا به ۱ و read_count به ۰ مقداردهی می‌شوند. سمافور rw_mutex میان پردازه‌های خواننده و نویسنده مشترک است. سمافور mutex برای تضمین انحصار متقابل در موقع به روزکردن متغیر read_count استفاده می‌شود. متغیر read_count، رد تعداد پردازه‌هایی را نگه می‌دارد که فعلاً به خواندن شئ مشغول هستند. سمافور rw_mutex، نقش سمافور انحصار متقابل را برای نویسندگان ایفا می‌کند. همین‌طور، توسط اولین یا آخرین خواننده‌ای که به بخش بحرانی وارد یا از آن خارج می‌شود، استفاده می‌گردد. اما به کار خوانندگانی نمی‌آید که در زمان ورود یا خروج خود، خوانندگان دیگری را در حال اجرای بخش‌های بحرانی خود می‌بینند.

کد پردازه‌ی نویسنده در شکل ۷–۳ و کد پردازه‌ی خواننده هم در شکل ۷–۴ نشان داده می‌شوند. توجه کنید که، اگر نویسنده‌ای در بخش بحرانی بوده و n خواننده هم منتظر باشند، در این صورت یکی از خواننده‌ها در صف rw_mutex و n – ۱ خواننده‌ی دیگر در mutex به صف می‌ایستند. همچنین دیده می‌شود که هر موقع، نویسنده‌ای signal(rw_mutex) را اجرا می‌کند، ممکن است که اجرای خوانندگان منتظر و یا فقط یک نویسنده‌ی منتظر را از سر بگیریم. انتخاب توسط زمانبند انجام می‌شود.

```
while (true) {
        wait(rw_mutex);
                . . .
        /* writing is performed */
                . . .
        signal(rw_mutex);
}
```

شکل ۷–۳ ساختار پردازه‌ی نویسنده.

مسأله‌ی خوانندگان–نویسندگان و راه‌حل‌های آن تعمیم داده می‌شود تا قفل‌های **خواننده–نویسنده** را در برخی سیستم‌ها فراهم کند. کسب یک قفل خواننده–نویسنده، مستلزم مشخص‌کردن مد قفل می‌باشد: دسترسی *خواندن* و یا دسترسی *نوشتن*. زمانی‌که پردازه‌ای فقط به خواندن داده‌های مشترک تمایل دارد، قفل خواننده–نویسنده را با مد خواندن درخواست می‌نماید؛ هر پردازه که بخواهد داده‌های مشترک را به روز نماید بایستی قفل را با مد نوشتن درخواست کند. در حالی‌که پردازه‌های متعددی می‌توانند قفل خواننده–نویسنده را با مد خواندن

دسترسی نمایند، تنها یک پردازه‌ی نویسنده می‌تواند قفل را برای نوشتن کسب کند، چرا که دسترسی انحصاری برای نویسندگان ضروری است.

قفل‌های خواننده–نویسنده در شرایط زیر فوق‌العاده مفیدند:

- در آن دسته از برنامه‌های کاربردی که شناسایی این که، کدام پردازه‌ها فقط خواننده‌ی داده‌های مشترک و کدام پردازه‌ها فقط نویسنده‌ی داده‌های مشترک هستند، آسان است.
- در آن دسته از برنامه‌های کاربردی که تعداد خوانندگان از نویسندگان بیشتر است. دلیل آن این است که سربار قفل‌های خواننده–نویسنده عموماً نسبت به برپایی سمافورها یا قفل‌های انحصار متقابل بیشتر است. افزایش همروندی حاصل از داشتن خوانندگان متعدد می‌تواند سربار مشمول برپایی قفل خواننده–نویسنده را جبران کند.

```
while (true) {
        wait(mutex);
        read_count++;
        if (read_count == 1)
                wait(rw_mutex);
        signal(mutex);
                . . .
        /* reading is performed */
                . . .
        wait(mutex);
        read count--;
        if (read_count == 0)
                signal(rw_mutex);
        signal(mutex);
}
```

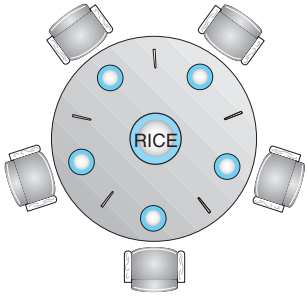
شکل ۷–۴ ساختار پردازه‌ی خواننده.

۷–۱–۳ مسأله‌ی فیلسوفان خوش‌خوراک

پنج فیلسوف را در نظر بگیرید که زندگی خود را به تفکر و خوردن سپری می‌کنند. فلاسفه پیرامون یک میز گرد مشترک با پنج صندلی نشسته‌اند. در وسط میز، دیس برنجی قرار گرفته و روی میز، پنج عدد چوبک[1] قرار دارند (شکل ۷–۵). زمانی که فیلسوفی به فکر فرو می‌رود با همکارانش تعاملی ندارد. هر از گاهی، یک فیلسوف گرسنه شده و سعی می‌کند که دو چوبکی را بردارد که به او نزدیکترند (چوبک‌هایی که بین خود و همسایگان چپ و راست خود می‌بیند). یک فیلسوف در هر لحظه می‌تواند تنها یک چوبک را بردارد. واضح است که او نمی‌تواند چوبکی را

1. Chopsticks

بردارد که همسایه‌اش پیشتر برداشته است. زمانی‌که یک فیلسوف هر دو چوبک خود را با هم به دست می‌آورد، بدون این که چوبک‌ها را رها کند مشغول خوردن می‌شود. لحظه‌ای که سیر می‌شود، هر دو چوبک را پایین گذاشته و دومرتبه به فکر فرو می‌رود.



شکل ۷-۵ وضعیت استقرار فیلسوفان خوش‌خوراک.

مسأله‌ی فیلسوفان خوش‌خوراک[1] به دلیل اهمیت کاربردی آن به عنوان یک مسأله‌ی کلاسیک همگامی مطرح است و نبایستی ساده‌لوحانه، این مسأله را احیاناً به حساب تنفر دانشمندان حوزه‌ی کامپیوتر از فلاسفه بگذاریم، این مسأله به واقع نمونه‌ای از رده‌ی بزرگ مسائل کنترل همروندی است و نمایش ساده‌ای از ضرورت تخصیص منابع متعدد مابین چندین پردازه به شیوه‌ای عاری از بن‌بست و قحطی‌زدگی می‌باشد.

۷-۱-۳-۱ راه‌حل سمافور

یک راه‌حل ساده آن است که هر یک از چوبک‌ها را با یک سمافور نشان بدهیم. هر فیلسوف سعی می‌کند تا یک چوبک را با اجرای عملیات wait() بر روی سمافور آن بردارد؛ او چوبک‌های خود را با اجرای عملیات signal() بر روی سمافورهای مناسب رها می‌سازد. بنابراین، داده‌های مشترک عبارتند از:

```
semaphore chopstick[5];
```

که در ابتدا تمامی عناصر chopstick به ۱ مقداردهی می‌شوند. ساختار فیلسوف i در شکل ۷-۶ نشان داده می‌شود.

اگرچه این راه‌حل تضمین می‌کند که هیچ دو همسایه نمی‌توانند همزمان مشغول خوردن باشند، اما بایستی آن را رد کرد، چرا که می‌تواند باعث بن‌بست شود. فرض کنید که همگی فیلسوف‌ها در یک لحظه گرسنه می‌شوند و هر یک، چوبک سمت چپ خود را بر می‌دارند. اکنون تمامی عناصر chopstick برابر ۰ هستند. زمانی که فیلسوفی می‌خواهد چوبک سمت راست خود را بردارد، برای همیشه به تأخیر می‌افتد.

روش‌های متعددی برای پرهیز از مسأله‌ی بن‌بست به صورت زیر مطرح هستند:

1. Dinning-philosophers problem

- اجازه بدهید که همزمان، حداکثر چهار فیلسوف دور میز بنشینند.
- اجازه بدهید که هر فیلسوف فقط زمانی اقدام به برداشتن چوبک‌هایش نماید که هر دو موجودند (برای انجام این کار، بایستی آنها را در یک بخش بحرانی بردارد)
- از یک راه‌حل نامتقارن استفاده کنید یعنی، فیلسوف فرد نخست چوبک سمت چپ و سپس چوبک سمت راست خود را بردارد، در حالی‌که فیلسوف با شماره‌ی زوج ابتدا چوبک سمت راست و بعد از آن، چوبک سمت چپ خود را بردارد.

```
while (true) {
        wait(chopstick[i]);
        wait(chopstick[(i+1) % 5]);
                . . .
        /* eat for awhile */
                . . .
        signal(chopstick[i]);
        signal(chopstick[(i+1) % 5]);
                . . .
        /* think for awhile */
                . . .
}
```

شکل ۷-۶ ساختار فیلسوف i.

در بخش ۶-۷، راه‌حل جدیدی برای مسأله‌ی فیلسوفان خوش‌خوراک ارائه می‌هیم که عاری از بن‌بست بودن آن تضمین است. با این وجود، توجه داشته باشید که راه‌حلی برای مسأله‌ی فیلسوفان خوش‌خوراک مطلوب است که، اطمینان بدهد هیچ یک از فلاسفه، گرفتاری قحطی‌زدگی منجر به فوت نخواهند داشت. راه‌حلی که عاری از بن‌بست است، الزاماً احتمال وقوع قحطی‌زدگی را از بین نمی‌برد.

۷-۱-۳-۲ راه‌حل مانیتور

در ادامه، مفاهیم مانیتورها را با طرح یک راه‌حل عاری از بن‌بست برای مسأله‌ی فیلسوفان خوش‌خوراک روشن می‌کنیم. محدودیتی که این راه‌حل اعمال می‌کند این است که هر فیلسوف فقط در صورتی می‌تواند چوبک‌های خود را بر دارد که هر دوی آنها توأمان در دسترس باشند. چون هر یک از فلاسفه را در یکی از سه حالت ممکن می‌توان یافت، در نوشتن این راه‌حل لازم است این سه حالت را از همدیگر تمییز بدهیم. به همین منظور، ساختمان‌داده‌ای به شکل زیر معرفی می‌کنیم:

```
enum {THIKNIKG, HUNGRY, EATING} state[5];
```

فیلسوف i فقط در صورتی می‌تواند state[i] = EATING انتخاب کند که هر دو همسایه‌اش در حال خوردن نباشند یعنی:

```
(state[(i+4) % 5] != EATING) && (state[(i+1) % 5] != EATING)
```

همین‌طور لازم داریم که اعلانی به صورت زیر داشته باشیم:

```
condition self[5];
```

با این شیوه، فیلسوف i موقعی که گرسنه است، اما قادر به برداشتن چوبک‌های مورد نیازش نیست، می‌تواند خود را به تأخیر بیندازد.

هم‌اکنون آمادگی داریم تا راه‌حل خود را برای مسأله‌ی فیلسوفان خوش‌خوراک تشریح کنیم. توزیع چوبک‌ها توسط مانیتور DiningPhilosophers که تعریف آن در شکل ۷-۷ نشان داده شده است، کنترل می‌شود. هر فیلسوف، پیش از آن که شروع به خوردن کند بایستی عملیات pickup() را احضار نماید. این کار ممکن است موجب تعلیق پردازه‌ی فیلسوف گردد. فیلسوف با تکمیل موفقیت‌آمیز عملیات خود می‌تواند خوردنش را شروع کند. در ادامه، فیلسوف عملیات putdown() را احضار می‌کند. بنابراین، فیلسوف i بایستی عملیات pickup() و putdown() را به صورت دنباله‌ی زیر احضار نماید:

```
DiningPhilosophers.pickup(i);
                ...
                eat
                ...
DiningPhilosophers.putdown(i);
```

به راحتی می‌توان نشان داد که این راه‌حل تضمین می‌کند هیچ دو همسایه‌ای نمی‌توانند همزمان مشغول خوردن باشند و هیچ‌گونه بن‌بستی اتفاق نخواهد افتاد. با این وجود، توجه داریم که همچنان امکان مرگ یک فیلسوف در مواجهه با قحطی‌زدگی وجود دارد. راه‌حلی برای این مسأله ارائه نمی‌کنیم، بلکه آن را به عنوان یک تمرین برای شما واگذار می‌کنیم.

۷-۲ همگامی در میان هسته

در ادامه، راهکارهای همگامی را که توسط سیستم‌عامل‌های ویندوز و لینوکس فراهم می‌شوند، مطرح می‌کنیم. این دو سیستم‌عامل، مثال‌های خوبی از رویکردهای متفاوت همگامی هسته فراهم می‌کنند و همان‌گونه که خواهید دید راهکارهای همگامی موجود در این سیستم‌ها، تفاوت‌های ظریف و قابل توجهی دارند.

۷-۲-۱ همگامی در ویندوز

سیستم‌عامل ویندوز یک هسته‌ی چندنخی است که پشتیبانی برنامه‌های کاربردی بلادرنگ و